

Лабораторная работа №3. Асинхронное программирование. Promise

1. Прочитать! Разобраться! И понимать!:
<https://learn.javascript.ru/promise-basics>
<https://learn.javascript.ru/promise-chaining>
<https://learn.javascript.ru/async-await>

Ознакомиться:

<https://learn.javascript.ru/promise-error-handling>
<https://learn.javascript.ru/promise-api>
<https://learn.javascript.ru/microtask-queue>

Знать:

- синхронный и асинхронный код
- что такое промис
- состояния промиса
- методы промисов (then, catch, finally, all)
- цепочка промисов
- async/await
- Разница между Promise.all, Promise.allSettled, Promise.race и Promise.any.
- Понятие "Callback Hell" и как промисы его решают.

Практические задания

Задание 1. Симулятор "Службы доставки" (Promise Chaining)

Создайте цепочку функций, имитирующую заказ еды:

1. `checkStock(item)` — проверяет наличие (задержка 1 сек). Если товара нет — `reject`.
2. `processPayment(order)` — имитирует оплату (задержка 2 сек). Если на счету < 500 руб — `reject`.
3. `deliverOrder()` — имитирует доставку (задержка 1.5 сек). **Требование:** Используйте цепочку `.then()`. В конце обязательно добавьте `.finally()`, который выведет "Спасибо за заказ, приходите еще!".

Задание 2. Гонка запросов (Promise.race)

Создайте две функции: `fetchFast()` (задержка 500мс) и `fetchSlow()` (задержка 2000мс). Каждая возвращает строку со своим названием.

- Используйте `Promise.race`, чтобы получить результат того, кто выполнится быстрее.
- **Вопрос студенту:** В каких реальных ситуациях в вебе нам может понадобиться `Promise.race`? (Подсказка: таймаут запроса).

Задание 3. "Умный" агрегатор (Promise.allSettled)

Создайте массив из 5 промисов. Пусть 3 из них завершатся успешно, а 2 — с ошибкой.

- Используйте `Promise.allSettled`.
- Выведите в консоль только результаты **успешных** операций.
- *Почему в данном случае `allSettled` лучше, чем `all`?*

Задание 4. Логическая задача "Микрозадачи"

Что будет выведено в консоль и в каком порядке? Объясните работу очереди микрозадач.

```
console.log('Начало');

setTimeout(() => console.log('Таймаут'), 0);

Promise.resolve()
  .then(() => console.log('Промис 1'))
  .then(() => console.log('Промис 2'));

console.log('Конец');
```

Задание 5. Рефакторинг на Async/Await с обработкой ошибок

Дан код на `.then()`:

```
function getData() {
  fetch('https://api.example.com/data')
    .then(res => res.json())
    .then(data => console.log(data))
    .catch(err => console.error("Ошибка:", err));
}
```

Перепишите его на `async/await`. Обязательно используйте конструкцию `try...catch`. Добавьте проверку `if (!response.ok)` перед парсингом JSON.

Задание 6. Параллельный запуск с ограничением

Представьте, что вам нужно загрузить 10 картинок, но браузер позволяет делать только 3 запроса одновременно.

- Напишите функцию `limitRequests(urls, limit)`, которая будет выполнять асинхронные задачи пачками.

Вопросы:

- 1 Дайте определение асинхронности в контексте JavaScript.
- 2 Что представляет собой объект `Promise`? Опишите три его возможных состояния и правила перехода между ними (может ли промис сменить состояние из `fulfilled` обратно в `pending`?).
- 3 Какую роль играют аргументы `resolve` и `reject` в конструкторе промиса? Что произойдет, если вызвать оба этих метода внутри одного промиса?

- 4 В чем разница в назначении методов `.then()`, `.catch()` и `.finally()`? В каких случаях срабатывает каждый из них?
- 5 Объясните механизм «чейнинга». Что именно возвращает метод `.then()`? Как результат выполнения одного `.then()` попадает в следующий?
- 6 Что произойдет, если внутри `.then()` вернуть:
 - Простое число?
 - Другой объект `Promise`?
 - Ничего (`undefined`)?
- 7 Как работает «всплытие» ошибок в цепочке промисов? Если ошибка возникла в первом `.then()`, а `.catch()` стоит только в самом конце цепочки из пяти звеньев, будет ли она обработана?
- 8 Сравните `Promise.all()` и `Promise.allSettled()`. В какой ситуации использование `Promise.all()` может привести к потере данных, если один из запросов завершился неудачно?
- 9 Опишите сценарии использования `Promise.race()` и `Promise.any()`. Чем они отличаются при обработке ошибок (`reject`)?
- 10 Что на самом деле возвращает функция, помеченная ключевым словом `async`, даже если внутри нет явного `return new Promise()`?
- 11 Как работает оператор `await`? Что происходит с потоком выполнения функции, когда интерпретатор встречает `await`?
- 12 Как осуществляется обработка ошибок в блоках `async/await`? Почему важно использовать `try...catch` при работе с `await`?
- 13 Что такое очередь микрозадач (`Microtask Queue`)? Каким приоритетом она обладает по отношению к очереди макрозадач (`Macrotasks`, например, `setTimeout`)?
- 14 Опишите путь выполнения кода: синхронный стек -> микрозадачи -> рендеринг -> макрозадачи. Где в этой цепочке находятся промисы?